PROJECT #1

1.1. INTRODUCTION

In this project, you will design and simulate digital circuits using Verilog, apply simulation tools to verify functionality, and finally implement your design on an FPGA board. You will gain practical experience in structural coding, simulation debugging, and hardware implementation. The manual provides detailed steps and best practices to help you navigate each phase successfully.

1.2. OBJECTIVE

- **Develop Structural Verilog Coding Skills:** Translate schematic diagrams into structural Verilog code.
- **Simulation:** Use ModelSim to simulate digital circuits and analyze waveform outputs.
- **FPGA Implementation:** Integrate multiple modules (a binary multiplier and a 7-segment display decoder) and deploy the design using Quartus II on an FPGA board.
- **Report Preparation:** Document your process, code, simulation results, and design challenges in a well-structured report.

1.3. STRUCTURAL VERILOG AND LOGIC GATE SIMULATION

You will start by implementing the given logic unit schematic in Figure 1, using structural Verilog. This task emphasizes the translation of schematic logic into code and verifying the design with simulation.

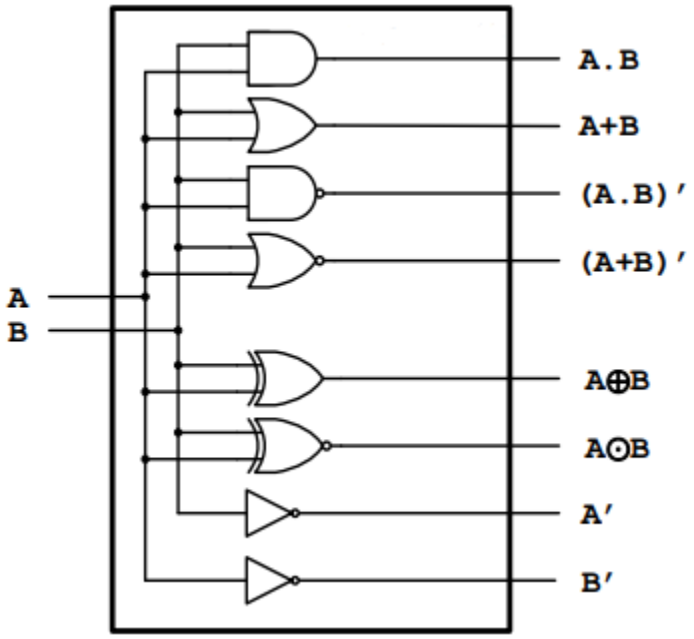

Figure 1: Logic Unit

Study the provided schematic carefully. Identify all the logic gates and their interconnections.

1.3.1. Write Structural Verilog Code

Create a new folder named project1 (do not include spaces). Open Modelsim and create a new project in this folder and name it project1a. Create a Verilog file (e.g., logic_unit.v) and write you structural code here. Ensure that each gate instance is properly connected as per the schematic.

Best Practice: Add descriptive comments above each module and instantiation to explain their functionality.

```
// Example: Instantiating an AND gate

module logic_unit(input a, input b, output y);

  and gate1(y, a, b); // AND gate instance: y = a & b

endmodule
```

1.3.2. Simulation with ModelSim

Create a testbench (e.g., logic_unit_tb.v) to apply test vectors to your design and add it to the project. Compile your design and testbench in ModelSim. Run the simulation and open the waveform viewer to capture the output. Save screenshots or export waveform data for your report.

1.4. TWO-BIT BINARY MULTIPLIER DESIGN AND SIMULATION

This task focuses on designing a two-bit binary multiplier using structural Verilog. The multiplier schematic is provided in Figure 2 below, and you will implement, simulate, and document its functionality.

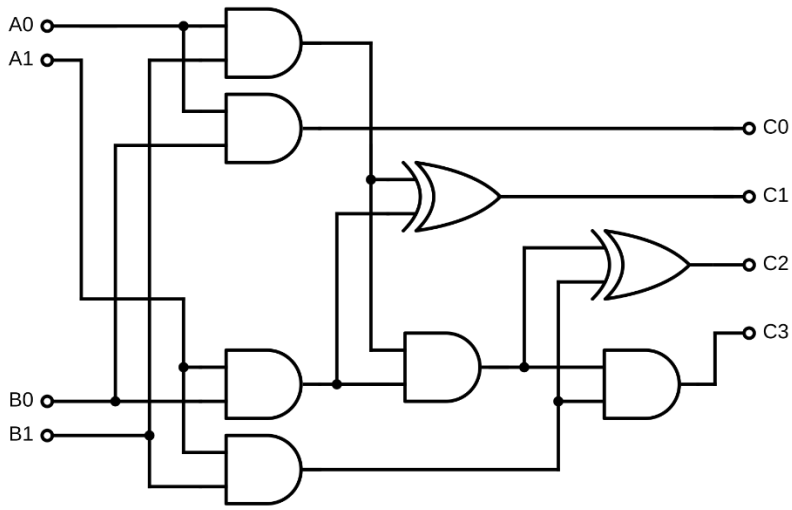

Figure 2: 2-bit binary multiplier

1.4.1. Multi-bit Signals

Many practical codes deal with buses instead of wires which are literally multibit ports or wires. In Verilog, to define the bus we use arrays. To define arrays, we simply add [most significant bit: least significant] before the name of the input, output or wires. The following are two examples:

```
output [0: 3] D;

wire [7: 0] SUM;
```

The first statement declares an output vector D with four bits, 0 through 3. The second declares a wire vector SUM with eight bits numbered 7 through 0. (Note: The first (leftmost) number (array index) listed is always the most significant bit of the vector.) The individual bits are specified within square brackets, so D [2] specifies bit 2 of D. It is also possible to address parts of vectors. For example, SUM [2: 0] specifies the three least significant bits of vector SUM.

1.4.2. Develop the Structural Verilog Code

Still in the project1 folder, create a new project and call it project1b. Create a new Verilog file here (e.g., binary_multiplier.v), and define the module interface with two 2-bit inputs and the corresponding 4-bit output. Instantiate sub-modules as per the schematic.

Add comprehensive comments to explain the role of each instantiated module.

```
// Example snippet for partial product generation

module binary_multiplier(input [1:0] A, input [1:0] B,
output [3:0] C);

  wire p0, p1, p2, p3;

  and gate0(p0, A[0], B[0]);

  and gate1(p1, A[0], B[1]);

  and gate2(p2, A[1], B[0]);

  and gate3(p3, A[1], B[1]);


  // Further addition logic follows to sum the partial
products

endmodule
```

1.4.3. Simulate the Design in ModelSim

In the same project, create a testbench (e.g., binary_multiplier_tb.v) to provide various combinations of inputs. Compile and run the simulation, and capture waveform outputs that demonstrate how the multiplier works.

1.5. 7-SEGMENT DISPLAY DECODER AND FPGA IMPLEMENTATION

In the final task, you will design a 7-segment display decoder in Verilog. This decoder takes a 4-bit binary input and displays the corresponding numeral on the rightmost 7-segment display of the DE0 FPGA board. The output of your binary multiplier (from section 1.4) should be connected to the decoder.

1.5.1. Design the 7-Segment Decoder

Still in project1b, create a new Verilog file (e.g., seg_decoder.v). Define a module that maps each 4-bit binary number from 0-9 to the correct segments (0–6) for the display as seen in Figure 3.

Use a case statement or if-else ladder to cover all possible input combinations.

Remember each 7-Segment Display on DE0 board is characterized by 8 independent active low outputs (including the dot in the lower right) that need to be all defined to determine the displayed character. For this project, you will not use the DP, so you can ignore it in your design.

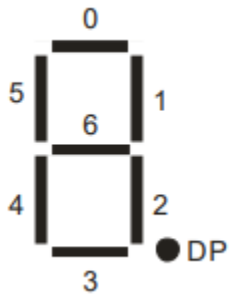

Figure 3: Position and index of each segment in a 7-segment display

```
// Example snippet for a 7-segment decoder
module seg_decoder(input [3:0] num, output reg [6:0] seg);
  always @(*) begin
    case(num)
      4'd0: seg = 7'b1000000; // Display 0
      4'd1: seg = 7'b1111001; // Display 1
      // Add remaining cases...
      default: seg = 7'b1111111; // Off
    endcase
  end
endmodule
```

1.5.2. Integration with the Multiplier

Still in project1b, create a top-level module (e.g., project1b.v) that instantiates both the binary multiplier and the 7-segment decoder. Connect the output of the multiplier to the decoder's input.

Provide detailed comments on how the integration works, emphasizing the data flow from the multiplier to the decoder.

Create a testbench (e.g., project1b_tb.v) to provide various combinations of inputs. Compile and run the simulation, and capture waveform outputs that demonstrate how the combination works.

```
// project1b.v
module project1b (
    input wire [1:0] A,
    input wire [1:0] B,
    output wire [6:0] seg
);
    wire [3:0] C;
    // Instantiate two-bit multiplier
    binary_multiplier mult (___, ___, ___);
    // Instantiate 7-segment decoder with multiplier output
    seg_decoder dec (___, ___);
endmodule
```

1.5.3. Multi-bit Signal Entry in Schematic Tool

Multi-bit signals are entered using Orthogonal Bus Tool. A signal is interpreted as multi-bit only after it is labelled. Some examples of valid multi-bit signals names are:

```
X [3..0] : A 4-bit signal named using vector notation
A, B, C, D : Four 1-bit signals grouped together to form a 4-bit signal
Y [4..2] : A 3-bit signal; the name implies it may be a sub-branch of the
Y signal with more bits
```

1-bit or multi-bit signals (or I/O ports) are sometimes combined into a multi-bit signal. Quartus II schematic tool requires that a multi-bit signal and each net connected to it through a bus are all clearly labelled. One clean way to do this is to right-click on the segment you need to name and select Properties. Below is an example of multi-bit signal connectivity in Figure 4.

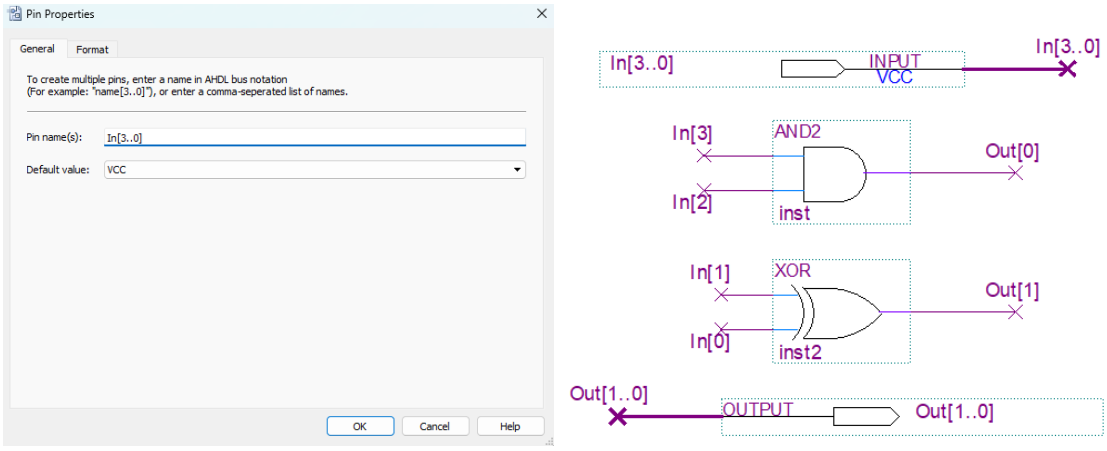

Figure 4: Multi-bit signal connectivity and naming example

1.5.4. FPGA Implementation with Quartus II

Open Quartus II and create a new project still in the project1 folder you created earlier. Do not forget to select the Cyclone III EP3C16F484C6 device.

Create a new Verilog file and copy and paste your Verilog files (binary_multiplier.v, seg_decoder.v, and project1b.v) in that respective order; one module after the other. Save it as project1b.v. Compile the project. If errors occur, consult the Quartus messages for troubleshooting.

Below is a code snippet of what it should look like in Quartus:

```
module binary_multiplier ();
    //code
endmodule

module seg_decoder ();
    //code
endmodule

module project1b ();
    //code
endmodule
```

Once the design has been fully completed, choose File from Top Menu, and roll the mouse over to Create/Update selection. A side menu appears, from this menu select Create Symbol Files for Current File in order to create a symbol (.bsf file) for your design.

Select File > New and choose Block Diagram/Schematic File and click OK. This opens the Graphic Editor window. The first step is to specify a name for the file that will be created. Select File > Save As. In the box labelled File name, type proj1. This will be the name of your schematic.

Double-click on the blank space in the Graphic Editor window. A pop-up window appears. Expand the hierarchy in the Project box and add the project1b schematic file. Use the same procedure for choosing input and output port symbols from the library primitives/pin. Assign names to the input and output port symbols as follows: Make sure nothing is selected by clicking on an empty spot in the Graphic Editor window. Point to the word pin_name on top of the input symbol and double-click the mouse. A dialogue box will appear. Type the pin name, and click OK.

Click on the wire icon in the toolbar to activate the Orthogonal Node Tool, the thicker wire is the bus tool for multi-bit signals. Position the mouse pointer over the right edge of one of your input pins. Click and hold the mouse button and drag the mouse to the right until the drawn line reaches the pinstub at the input of your scehmatic.

Connect all the I/Os and compile again.

Assigning Board Pins:

Pin assignments are made by using the Pin Planner. Select Assignments > Pin Planner to start. Select A[0] as the first pin to be assigned. To do this, double-click on the box in the column labelled Location to the right of A[0] entry. A drop-down menu appears. Scroll down and select PIN_J6. Instead of scrolling down the menu to find the desired pin, you can just type the name of the pin (J6) in the Location box and hit Enter.

Use the Pin Planner to assign:

- Two switches for each 2-bit input (A and B).
- The output from the decoder (seg[6:0]) to the rightmost 7-segment display on the DE0 board. Refer to the DE0 board's pin assignment documentation for the exact pin numbers. The necessary pin mappings are shown below:

Signal Name	FPGA Pin No.	Description
SW[0]	PIN_J6	Slide Switch[0]
SW[1]	PIN_H5	Slide Switch[1]
SW[2]	PIN_H6	Slide Switch[2]
SW[3]	PIN_G4	Slide Switch[3]

Figure 5: Pin assignments for the slide switches

Signal Name	FPGA Pin No.	Description
HEX0_D[0]	PIN_E11	Seven Segment Digit 0[0]
HEX0_D[1]	PIN_F11	Seven Segment Digit 0[1]
HEX0_D[2]	PIN_H12	Seven Segment Digit 0[2]
HEX0_D[3]	PIN_H13	Seven Segment Digit 0[3]
HEX0_D[4]	PIN_G12	Seven Segment Digit 0[4]
HEX0_D[5]	PIN_F12	Seven Segment Digit 0[5]
HEX0_D[6]	PIN_F13	Seven Segment Digit 0[6]

Figure 6: Pin assignments for the 7-segment displays

After pin assignment, compile you project again.

Note: While the majority of the project work is done at home, you are welcome to attend office hours to test your code on physical FPGA boards.

1.6. PROJECT REPORT REQUIREMENTS

Your final report should be well-structured and include the following sections:

Title Page

- Project title, your name, course, and date.

Objective Statement

- A brief description of what you aimed to achieve in the project.

Introduction

- Overview of the key concepts (structural Verilog, simulation, and FPGA implementation).

Body

- Verilog Code: Include all your source code (properly commented) for the logic unit, multiplier, and decoder.
- Simulation Results: Provide screenshots of waveform outputs from ModelSim along with explanations of what each waveform demonstrates.
- Quartus II Schematics: Include screenshots of the Quartus II environment (schematic view, pin assignments, etc.) that show how the full project is integrated.

Conclusion

- Summarize your findings, challenges faced, and how you resolved them.
- Reflect on the integration process and any potential improvements for future projects.

1.7. REFERENCES

- DE0 Board User Manual, Terasic Technologies Inc.
- Digital Design with An Introduction to the Verilog HDL, 5th Edition, M. Morris Mano & Michael D. Ciletti, Pearson
- Quartus II Introduction Using Schematic Design, ALTERA.